TiberCAD C++ Style Guidelines

1 Formatting

1.1 indentation

- Don't use tabs for indentation. Configure your editor to convert tabs into spaces.
- Use 2 spaces for indentation (can be configured in editor).
- Don't indent braces. They should be on the same indentation level as the corresponding keyword:

```
if (i == 0)
{
  ...
}
```

1.2 spaces / lines

- Use single spaces to separate keywords, braces, parentheses, operators:

```
for (...)
switch (...)
int value = 2 * sqrt(a * a + b * b);
static double d = (i < j) ? pi : pi / 2.0;
```

- Don't insert spaces before parentheses of function calls, before or after ".", "->" operators and "::" and "<"/">" in templates :

```
exp(...)
myobject.get_weird_value()
this->do_something()
a[i]
object(n)
std::string s = MyClass::get_name()
```

- Insert (one or more) blank lines to separate blocks of code, functions etc.
- Try to avoid long lines. Limit line length to about 80-85 characters. Wrap long lines in a logic way, split long expressions into smaller subexpressions, indent the wrapped lines. (This rule should help readability, enables nice printing, and if someone for some reason has to work on a text console he probably cannot scroll horizontally...)

1.3 brace placement

- Place braces alone on a new line:

```
if (i == j)
{
  ...
}
else
{
  ...
}

class MyClass
{
  ...
};

void function(void)
{
}
```

- do not indent braces
- omit braces only if code remains easily understandable

2 Naming

2.1 general

- use meaningful names: prefer “time” over “t”, “index” over “i” unless it is a temporary variable with clear purpose (“i” in loops etc.)
- use complete words: prefer “long_range_interaction_parameter” over “lr_int_par”, “set_signal_number” over “set_sig_nr”

2.2 types, classes, enumerations

- Use nouns to name classes, enumerators and typedefs
- Use uppercase for enumeration constants
- Use the scheme “MySpecialClass” for naming

```
enum Statistics
{
  BOLTZMANN,
  FERMIDIRAC
};
```

```
class RiverBoat;
typedef RiverBoat Boat;
```

2.3 functions, variables

- use verbs to name functions. Use “get”, “set”, “is” for accessor functions to class members
- use nouns to name variables, unless something else makes more sense
- use the scheme “my_nice_variable” to name variables and functions

```
int iterations;
double relative_error;
double get_relative_error(void);
void set_name(const std::string name);
bool is_initialized(void);
```

3 Classes, structs

3.1 general

- prefer classes over structs. Use structs only inside classes to define containers to logically combine variables etc.
- try to keep classes simple, i.e. try to not put too much functionality into one single class
- do *not* use friendship, unless in embedded classes, if it makes sense, or in stream operators when they are not class members
- make constructor protected if the class should not be instantiated directly
- use initializer list in constructors, initialize all variables, initialize all pointers to NULL

3.2 class structure

- declare class members in the order public, protected, private
- declare first nested classes, structs, enumerations, then typedefs, then functions, then variables
- don't put implementations into the class declaration unless they are very short (1 line). Put them after the class declaration

3.3 member variables

- make all member variables private and provide public or protected accessor functions, unless you have good reasons
- use a special naming for private member variables to distinguish them from ordinary variables, e.g. _my_variable, my_variable_, t_my_variable (but do not use leading underscore with one letter variables)
- initialize member variables in the order they are declared

3.4 member functions

- make all member functions private that are not intended to be called from outside the class
- name all function parameters with meaningful names
- use default arguments in functions and constructors

```
void set_name(const std::string& name,
              bool is_nickname = false)
{
  _name = name;
  ...
}
```

- don't inline member functions that have more than about ten lines of code
- keep member functions “const correct”: declare arguments “const” if they are not to be changed by the function (unless they are simple data types int, double etc.), declare the function “const” if it does not change object state, return const references to class member of non simple types

```
const std::string& get_name(void) const;
void set_name(const std::string& name);
```

- if possible put only one return statement at the end of a non-void function. If you use return statements inside the function, comment them in a clear way. In any case include a return statement at the end to prevent annoying.
- you can use the function ignore_unused_variable(..) in TypeDefs.h to get rid of compiler warnings about unused variables.
- do not put unnecessary semicolons after function implementations (compiler compatibility problems)
- comment return values and arguments, possible pre- and postconditions

3.5 h-files, C-files

- use extensions .h, .C
- use a “one-class-per-file” approach and name the file exactly as the class it contains
- write as first line `// $Id$` and run `svn propset svn:keywords` on new files. (this can be automatized in the subversion configuration file)
- Put everything in an .h file inside a

```
#ifndef _FILENAME_H_
#define _FILENAME_H_
your code here
#endif // _FILENAME_H_
```

- comment your code. Use doxygen commenting style in .h files. Comment everything that might

be not immediate for somebody else than yourself. Have in mind that sooner or later someone else will have to maintain your code, if TiberCAD will be successful (which we strongly desire, don't we?) Especially comment whenever you deviate from guidelines in a non-obvious way.

- use as few as possible include statements in header files. Include only the necessary. Use forward declarations whenever possible (use object pointers and references in class declaration)

```
class MyClass;  // not include "MyClass.h"
MyClass* _my_class;
```

- do *not* use `using namespace XXX` in header files

3.6 inheritance / templates

- use templates with care
- use inheritance when there is an "is a" relationship between classes, not in other cases (eg. "has a" relationship)

```
class Fruit
class Core
class Apple : public Core  // NO: apple only has a core
class Apple : public Fruit  // ok: apple is a fruit
{
  private:
    Core _core;  // ok: apple has a core
};
```

- declare virtual methods as protected and provide public non-virtual methods which call the virtual ones (and can perform eg. some common tasks)
- do not override non-virtual methods in derived classes
- always declare a virtual destructor in classes that can be inherited
- do not use multiple inheritance if you do not have very good reasons for doing so
- in derived constructors call constructor of base class (in initializer list)
- declare member variables private even if they have to be accessed by derived classes. Provide protected accessor methods

3.7 general considerations

- avoid global functions, enums etc. Encapsulate them in namespaces or (better) classes
- use C++ style include statements (eg. <math>, not <math.h>)
- handle pointers with care. After deleting a pointer, assign NULL to it.
- if possible reuse objects instead of recreating them to avoid reallocation
- If you need global variables or functions in a file, declare them in an unnamed namespace